

Roll Number

--	--	--	--	--	--	--

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking



Partially Filled



Lightly filled



Analysis & Design of Algorithms 2026 — Quiz 3

Duration: 1 hour

Total Marks: 20

Name: _____

Roll No.: _____

Problem 1. A *subsequence* of a string is obtained by deleting zero or more characters without changing the order of the remaining characters. For example, **eph**n is a subsequence of **elephant**. You are given a string A of length n and another string B of length m ($m \leq n$). For each position $j \in \{1, \dots, n\}$, matching the j -th character of A incurs a cost p_j . Your goal is to determine the *minimum possible cost* of selecting a *non-empty* subsequence of A that equals B . The total cost of a subsequence is defined as the sum of the costs of the matched positions. You may assume that a non-trivial feasible solution always exists. Your algorithm should run in time $O(mn)$.

Example. Let $A = \text{bacbcbabcacba}$, $B = \text{bcbca}$, and suppose the costs for the characters of A are

$$(2, 5, 3, 1, 2, 5, 3, 1, 3, 1, 2, 4, 1).$$

One possible matching of B in A uses the positions 1, 3, 4, 5, 7, giving total cost $2 + 3 + 1 + 2 + 3 = 11$. However, a cheaper matching uses the positions 4, 5, 8, 11, 13, giving total cost $1 + 2 + 1 + 2 + 1 = 7$. Thus, the minimum cost of matching B as a subsequence of A in this example is 7.

(a) Define the subproblems for dynamic programming

Subproblem: $\text{cost}(i, j)$ = The minimum cost to match first j character of B as a subsequence of the first i character of A . Here, $j \leq i \leq n$ and $0 \leq j \leq m$.

Rubric: +2 Marks (Requires correct definition of the subproblem).

(b) Write the recurrences for optimal substructure that handles all cases, including the base.

Base:

For $i = j = 1$

$$\text{cost}(1, 1) = \begin{cases} p_1 & \text{if } A[1] = B[1] \\ \infty & \text{else} \end{cases}$$

For $1 < i \leq n$ and $j = 1$,

$$\text{cost}(i, 1) = \begin{cases} \min\{\text{cost}(i-1, 1), p_i\} & \text{if } A[i] = B[1] \\ \text{cost}(i-1, 1) & \text{else} \end{cases}$$

For $i = j$ and $1 < j \leq m$,

$$\text{cost}(i, i) = \begin{cases} \text{cost}(i-1, i-1) + p_i & \text{if } A[i] = B[i] \\ \infty & \text{else} \end{cases}$$

Recurrence:

For $j < i \leq n$ and $1 < j \leq m$

$$\text{cost}(i, j) = \begin{cases} \min\{\text{cost}(i-1, j), \text{cost}(i-1, j-1) + p_i\} & \text{if } A[i] = B[j] \\ \text{cost}(i-1, j) & \text{else} \end{cases}$$

Final Solution: $\text{cost}(n, m)$

Rubric: +2 mark (correct base cases), +2 marks (main recurrence logic). -1 (if range not defined in base cases). -1 (if range not defined in the recurrence). -0.5 (if final solution is not stated correctly).

- (c) Write the pseudocode on the basis of the above recurrence. Mention inputs, outputs and data-structures (like array etc.) properly

Algorithm 1 Minimum Cost Subsequence

Require: String $A[1 \dots n]$, String $B[1 \dots m]$, Cost array $p[1 \dots n]$

Ensure: Minimum possible cost $\text{cost}(n, m)$

```
1: Initialize: 2D array  $\text{cost}[1 \dots n][1 \dots m]$ 
2: {— Base Case: For  $i = j = 1$  —}
3: if  $A[1] == B[1]$  then
4:    $\text{cost}(1, 1) \leftarrow p_1$ 
5: else
6:    $\text{cost}(1, 1) \leftarrow \infty$ 
7: end if
8: {— Base Case: For  $1 < i \leq n$  and  $j = 1$  —}
9: for  $i = 2$  to  $n$  do
10:   if  $A[i] == B[1]$  then
11:      $\text{cost}(i, 1) \leftarrow \min(\text{cost}(i - 1, 1), p_i)$ 
12:   else
13:      $\text{cost}(i, 1) \leftarrow \text{cost}(i - 1, 1)$ 
14:   end if
15: end for
16: {— Base Case: For  $i = j$  and  $1 < j \leq m$  —}
17: for  $i = 2$  to  $m$  do
18:   if  $A[i] == B[i]$  then
19:      $\text{cost}(i, i) \leftarrow \text{cost}(i - 1, i - 1) + p_i$ 
20:   else
21:      $\text{cost}(i, i) \leftarrow \infty$ 
22:   end if
23: end for
24: {— Recurrence: For  $j < i \leq n$  and  $1 < j \leq m$  —}
25: for  $j = 2$  to  $m$  do
26:   for  $i = j + 1$  to  $n$  do
27:     if  $A[i] == B[j]$  then
28:        $\text{cost}(i, j) \leftarrow \min(\text{cost}(i - 1, j), \text{cost}(i - 1, j - 1) + p_i)$ 
29:     else
30:        $\text{cost}(i, j) \leftarrow \text{cost}(i - 1, j)$ 
31:     end if
32:   end for
33: end for
34: return  $\text{cost}(n, m)$ 
```

Rubric: +0.5 mark (initialization), +1.5 mark (correct loop structure and instructions inside it).

- (d) Argue runtime of the above pseudocode

Solution: The dimension of the 2D array $\text{cost}()$ is $n \times m$. Hence, there are $O(nm)$ many subproblems to be solved. The base case initialization takes $O(n)$. Now, for the remaining subproblems, each of them are solved exactly once and executes not more than 7 instructions. So, it is $O(nm)$. Hence, overall time complexity is $O(nm)$.

Rubric: +1 mark (correct final complexity), +1 mark (justification).

Problem 2. You are given an array of n integers $A = (A_1, A_2, \dots, A_n)$, where the entries may be positive, zero, or negative. You are also given a positive integer K . A subsequence of indices $i_1 < i_2 < \dots < i_t$ where $t \leq n$ is said to be K -spaced if $i_{j+1} - i_j \geq K$ for all $j = 1, \dots, t-1$. The value of such a subsequence is defined as $A_{i_1} + A_{i_2} + \dots + A_{i_t}$. Design a DP based algorithm that runs in $O(n)$ time to compute the maximum possible value of a K -spaced subsequence of A where K is a constant. The subsequence must contain at least one element.

(a) Define the subproblems for dynamic programming

Subproblem: $\text{cost}(i)$ = The maximum sum of K -spaced subsequence in the first i character of A . Here, $0 \leq i \leq n$.

(b) Write the recurrences for optimal substructure that handles all cases, including the base.

Rubric: +2 Marks (Requires correct definition of the subproblem).

Base:

For $i = 1$, $\text{cost}(1) = A_1$

For $2 \leq i \leq K$, $\text{cost}(i) = \max\{\text{cost}(i-1), A_i\}$.

Recurrence:

For $K < i \leq n$

$$\text{cost}(i) = \max\{\text{cost}(i-1), A_i + \text{cost}(i-K), A_i\}$$

Final Solution: $\text{cost}(n)$

Rubric: +2 mark (correct base cases), +2 marks (main recurrence logic). -1 (if range not defined in base cases). -1 (if range not defined in the recurrence). -0.5 (if the final solution is not stated correctly).

(c) Write the pseudocode on the basis of the above recurrence. Mention inputs, outputs and data-structures (like array etc.) properly

Rubric: +0.5 mark (initialization), +1.5 mark (correct loop structure and instructions inside it).

(d) Argue runtime of the above pseudocode

Solution: The dimension of the 1D array $\text{cost}()$ is $(n+1)$. Hence, there are $O(n)$ many subproblems to be solved. Each of these subproblems are solved only once using constant many instruction. Hence, overall time complexity is $O(n)$.

Rubric: +1 mark (correct final complexity), +1 mark (justification).

Algorithm 2 Maximum Value K-Spaced Subsequence

Require:

0: Array A of n integers ($A[1 \dots n]$), Integer K ($1 \leq K \leq n$)

Ensure: Maximum value of a K -spaced subsequence

1: Data Structure:

2: Create array $\text{cost}[1 \dots n]$

3: **{1. Initialization & Base Cases}** {Handle case where all numbers are negative}

4: $\text{cost}(1) \leftarrow A_1$

5: **for** $i = 2$ **to** K **do**

6: $\text{cost}(i) \leftarrow \max\{\text{cost}(i-1), A_i\}$ {For first K , just pick max element so far}

7: **end for**

8: **{2. Fill Table for** $i > K$ **}**

9: **for** $i = K + 1$ **to** n **do**

10: $\text{skip} \leftarrow \text{cost}(i-1)$ {Option 1: Don't include A_i }

11: $\text{extend} \leftarrow A(i) + \text{cost}(i-K)$ {Option 2: Include A_i }

12: $\text{fresh} \leftarrow A(i)$ {Option 3: we start fresh if history is negative}

13: $\text{cost}(i) \leftarrow \max\{\text{skip}, \text{extend}, \text{fresh}\}$

14: **end for**

15: **{3. Return Result}**

16: **return** $\text{cost}(n)$
